MXET 300
Mechatronics I - Mobile Robotic Systems



Multidisciplinary
Engineering Technology
COLLEGE OF ENGINEERING

Final Project

Soda Retriever

Submission Date: 04/28/2026

Section: 504
Team Number: 13
Team members: Niamh Coles, Vivian VanCleve, Ben Herbst, Zavier Dollaway

Introduction:

This lab is meant to create a system in which the SCUTTLE robot identifies a distinct soda can and is able to pick it up and return it to the user. This project builds on the foundational skills learned throughout the semester, such as sensor calibration, data logging, Node-RED visualization, motor control, and embedded programming, by integrating them into an autonomous robotic system.

The motivation for this project arises from combining two core mechatronics challenges: precise manipulation using servo-driven mechanisms, and reliable autonomous navigation using LiDAR-based obstacle avoidance and camera-based visual alignment. These two subsystems were developed in parallel by separate sub-teams before being integrated onto the SCUTTLE chassis for final demonstration.

The project required the team to design and fabricate a vertical linear actuator and claw-like gripper actuated by MG995R servo motors controlled via a PCA9685 I2C servo driver; to implement LiDAR-based obstacle detection and avoidance for autonomous navigation to predetermined locations; and to use computer vision for identifying soda cans and aligning the robot with the drink. A Node-RED user interface was developed and the entire system was powered and managed using the SCUTTLE's existing battery with an external buck converter for servo power requirements.

This report describes the hardware and software design of each subsystem, the integration process, the testing methodology, and the results observed.

Procedure:

Hardware Overview

The SCUTTLE Soda Retriever system was built on the standard SCUTTLE v3 mobile robot platform, which uses a Raspberry Pi 4 as its primary computing unit and an omni-directional three-wheeled chassis for mobility. The hardware additions for this project are listed in the table below.

Component	Description / Role
SCUTTLE v3 Chassis	Base mobile platform with Raspberry Pi 4, three omni-wheels, and onboard LiDAR.
MG996R Servo Motors (x4)	High-torque servo motors used to actuate the claw-like gripper and the gripper pitch control.
PCA9685 I2C Servo Driver	16-channel PWM servo driver communicating with the Raspberry Pi over I2C to control all servo motors independently.
Buck Converter	Steps down SCUTTLE battery voltage to 5–6 V to safely power the servo motors.
USB Camera	Mounted on the front of the SCUTTLE for computer vision: soda can color detection.
LiDAR (RPLiDAR A1)	Provides 270° distance scanning for real-time obstacle detection and avoidance during autonomous navigation.
3D-Printed Arm Frame	Custom-designed vertical linear actuator housing and gripper claw printed in PLA for lightweight structural support.
External Battery Pack	Supplementary power supply for the servo system when SCUTTLE battery alone is insufficient.

Table 1: Hardware Components of the SCUTTLE Soda Retriever

Robotic Arm Subsystem

The robotic arm consists of two primary mechanisms: a claw-like gripper for grasping soda cans, and a gripper pitch mechanism to lift the can off of the ground.

The gripper pitch mechanism is used to lift the soda can off of the ground to enable the SCUTTLE to continue on its path. Two MG996R servos provide the lifting force. PWM pulse widths were mapped to level and lifted angular positions to ensure the can is lifted without expending too much power from the batteries.

Power for the servos was supplied through a buck converter set to 5.5 V connected to the SCUTTLE battery. Current draw during simultaneous actuation of all four servos was measured at approximately 1.6 A, within the converter's rated output.

NAVIGATION SUBSYSTEM

The navigation consists of two major components. First, the SCUTTLE uses computer vision to check if the soda can is within vision. If it is not, the SCUTTLE utilizes voxelization of the lidar data to intelligently search the room. The voxelization loop begins by populating local voxels with 'empty' values of 1. This indicates to the robot that this square is safe to travel to. These values are then overwritten by a lidar reading, which places every lidar reading in a radius of ~1 meter in a voxel, and populates that voxel's value as 2, or 'full'. Once the local voxels are populated, the robot randomly generates paths to the nearest unknown voxel, (value of 0), of minimal length until it finds a path that does not encounter any obstacles. If no minimal length path is found, the length of the path is increased until a path is found. This step also integrates a function that gives the robot a width of three voxels to avoid any collisions, (in this case, a voxel is 15cm x 15cm). Once a minimal path has been planned, it is then further shortened by eliminating any redundant steps. E.g. "step forward, and then step forward", would be reduced to "step forward for two units". This is done by starting at the end voxel destination, and seeing how many voxels before that it can skip without adding a collision to the path. It repeats this step with the last reachable voxel until the initial voxel has been reached, and saves the new path.

Once this optimized path has been generated, the path is executed as follows:

1. The robot turns from its current absolute heading, (read from the compass), to face the center of the target voxel, (the next voxel in the path). Change in heading is saved to 'globalVar', a variable that stores the robot's global position at all times.
2. The robot moves forwards, utilizing feedback from both the compass and encoders to get updated information on its current position, which it saves to globalVar.
3. Once the robot reaches its target voxel, it rescans for the soda can. If the soda can is present, the robot exits its path tracking and moves towards the can.
4. If no can is present, the robot continues on its path. When the end of the path is reached, the robot re-evaluates, finding the new nearest reachable "unknown" voxel, and generating a new path.

This method allows for a strategic searching of a space, while avoiding collisions, (dynamically, as it constantly checks for new obstacles after executing every small step). In cases where the soda can is not immediately visible to the robot, the robot should be able to search around and behind obstacles until it either finds the can, or runs out of reachable "unknown" spaces. **Figure 1** shows an example of the path formation to track to the nearest unknown voxel:

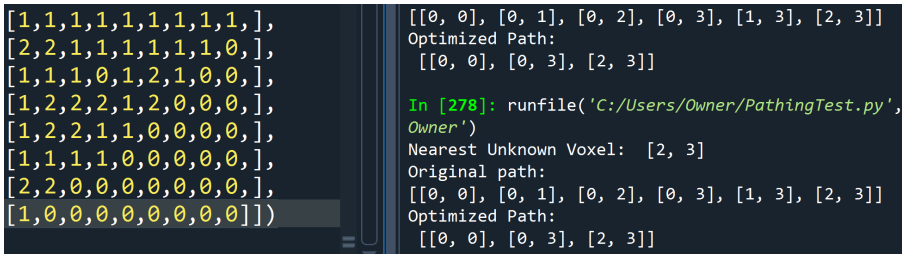


Figure 1. SCUTTLE Path Generation to Nearest Unknown Voxel

Figure 2 shows the lidar voxelized map, with the robot located at R. Filled squares indicate voxels with values of 2. The dots indicate empty units, and blank spaces indicate unknowns. The filled object accurately represent the surroundings, as well as the distance to each object.

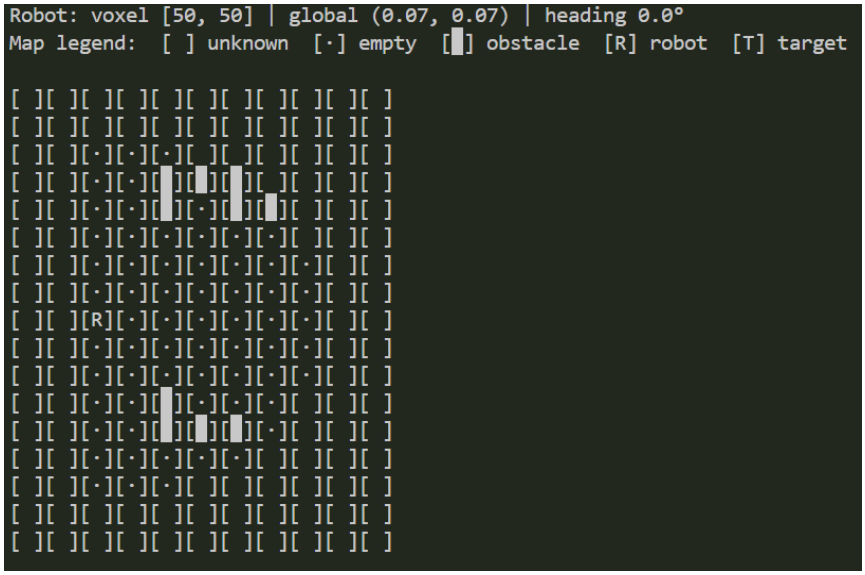


Figure 2. SCUTTLE LIDAR Voxelization

SODA CAN COLOR DETECTION

The color detection uses computer vision to detect and approach the desired can in real-time using a camera, HSV color filtering, and PI motion control. The system works by continuously capturing frames from the camera, converting to HSV color space, and applying filtering to isolate the regions matching the color of the desired can. The program then computes the horizontal angle of the object relative to the camera center and the pixel width of the object.

These measurements are fed into two PI controllers that regulate the motion of the robot. The angle controller will adjust the rotational velocity to ensure that the can is centered in the field of view of the camera, while the width controller will adjust the forward velocity of the robot to reach a certain distance from the can. When the object is not detected, the robot will enter its search mode by spinning and scanning for the object. When the object is detected, it will prioritize alignment before approaching until the target pixel width is reached. Once the can is at

the desired angle and pixel width for five consecutive frames, the program will stop and prepare to grab the object. The motor outputs used a clamped duty cycle to avoid issues with the duty cycle being too low to move the wheels.

Conclusion:

The SCUTTLE Soda Retriever project successfully demonstrated the integration of mechanical design, embedded control, computer vision, and autonomous navigation into a cohesive mechatronic system.
Each subsystem, the servo-actuated robotic arm, the LiDAR-based obstacle avoidance, the OpenCV color detection pipeline, and the Node-RED user interface, was developed, tested, and integrated iteratively over the four-week project period.

The major flaw in the whole system was the automatic room search function. The motors had a significant difference in motion regardless of speed, which caused the robot to drift significantly. This error was attempted to be corrected by reading the encoders and compass frequently, but since the magnitude of the error was so high, the robot drifted and caused the lidar readings to overwrite themselves in incorrect places. While the robot was able to follow a simple path, and detect obstacles, fully autonomous room searching was beyond it, as it was unable to retain an accurate and complete map of its surroundings due to this overwriting.

Overall, the project provided a comprehensive capstone-like experience that integrated the hardware and software skills developed throughout the semester into a functional autonomous robot.

Distribution of Work:

Ben Herbst	Computer vision can detection, report, slides
Vivian VanCleve	Robotic gripper CAD, Robotic gripper code, report, slides
Zavier Dollaway	Robotic gripper CAD, Robotic gripper code, report, slides
Niamh Coles	Lidar voxelization, path generation, report, slides

Appendix:

[Github link]

Lab 8 Formulas/ Commands:

bash start_mjpg_streamer_pi.sh L3_image_filter.py *// Starts the video filter script with the streamer as an input*

Lab 7 Formulas/ Commands:

N/A *// All commands used in the lab were used in previous labs*

Lab 6 Formulas/ Commands:

N/A *// All commands used in the lab were used in previous labs*

Lab 5 Formulas/ Commands:

sudo watch -n0.2 i2cdetect -y -r 1 *// Runs a program to detect and display currently connected I2C addresses.*

Lab 4 Formulas/ Commands:

$$\phi_L = \frac{\dot{x} - \frac{L}{2}\dot{\theta}}{r} \quad \text{Eq. 1}$$

$$\phi_R = \frac{\dot{x} + \frac{L}{2}\dot{\theta}}{r} \quad \text{Eq. 2}$$

Lab 3 Formulas/ Commands:

N/A *// All commands used in the lab were used in previous labs*

Lab 2 Formulas/ Commands:

mkdir directory_name *// Command to create a new directory in the working directory, called "directory_name".*

wget URL_OF_WEB_PAGE *// Retrieves content from the web at the URL provided*

python3 file_name.py *// Runs python script in the current directory called file_name.py*

sudo systemctl stop oled.service *// Stops the default OLED service script to prevent conflicts with the OLED script run during lab*

sudo mv <new file name> <file location to replace> *// Replaces a file at a specified location with a new file.*

sudo systemctl restart oled.service *// Restarts default OLED service script*

touch file_name.py *// Creates a file in the current directory with the name and type specified.*

Lab 1 Formulas/Commands:

gh auth login *// Command to begin signing in to a GitHub account.*

git config --global user.name “USERNAME” *// Configures the git account username locally*

git config --global user.email “EMAIL” *// Configures the git account email address locally*

git clone THE URL TO YOUR REPOSITORY *// Creates a local clone of the repository at the url. This is done after running the above commands to sign in.*

cat README.md *// Displays the entire contents of the README file.*

git add . *// Stages all changes to the repository to be later committed and pushed to main*

git commit -m “first commit message” *// Commits all changes as a new version locally. Stores version message to be saved with the new version.*

git push *// Pushes all local changes to the online GitHub repository, stored as the committed version with the version message.*